

ALEMBIC, PROPERLY UNDERSTOOD

Mastering Alembic

Database migrations in Docker and on local databases — built straight from the current Alembic documentation, with hands-on checkpoints instead of passive reading.

Prepared for Gelan

Covers Alembic 1.19.x · SQLAlchemy (sync & async) · SQLite & PostgreSQL · Docker Compose

Compiled August 2026

0. Before You Start

What this guide assumes about you: you already know Python OOP, you’ve built FastAPI + SQLAlchemy apps, and you’ve used Alembic *some* already inside TaskFlow (Docker + Postgres + async SQLAlchemy 2.0). This isn’t a “what is a database” guide. It goes straight at the tool, explains the *why* behind each command, and then gives you unsolved exercises so you build the muscle memory yourself instead of copy-pasting.

How to use it: read a section, run the commands for real against a scratch project (instructions in section 1), then do the “Try It Yourself” box at the end of that section *before* moving on. The hints are intentionally incomplete — that’s the point.

Version note: this guide is written against Alembic **1.19.x** (the `pyproject` and `checkconstraint_byname` features referenced below were added in 1.16 and 1.19 respectively — if you’re on an older Alembic, `pip install -U alembic` first).

1. What Alembic Actually Does

Alembic is SQLAlchemy’s migration tool. It solves one problem: **your database schema needs to change over time, in a repeatable, ordered, reversible way, across every environment that runs your app** (your laptop, your teammate’s laptop, staging, production, a Docker container that gets rebuilt from scratch every CI run).

Two things make this hard without a tool:

- If you just hand-edit tables locally, nobody else’s database matches yours.
- If you regenerate the schema from your models every time, you destroy existing data.

Alembic’s answer is a **linked list of small Python scripts** (“revisions”), each one saying how to go forward (`upgrade()`) and how to go back (`downgrade()`). A special table in your database, `alembic_version` , just stores *which revision the database is currently at*. Running `alembic upgrade head` walks forward from wherever you are to the newest revision, applying each one in order.

That’s the entire mental model. Everything else in this guide is detail on top of that.

Set up a scratch project

Do this now — you’ll use it for every exercise below.

```
mkdir alembic-lab && cd alembic-lab
python -m venv .venv && source .venv/bin/activate
pip install alembic sqlalchemy psycopg2-binary
```

2. The Migration Environment

Running `alembic init alembic` creates:

```
alembic-lab/
├── alembic.ini      # deployment config: DB URL, logging
├── alembic/
│   ├── env.py       # the script alembic actually runs
│   ├── script.py.mako # template for new revision files
│   └── versions/     # your revision scripts live here
```

`env.py` **is the important file**. It’s not boilerplate you can ignore — it’s a real Python script that runs every time you invoke `alembic`. It’s responsible for: getting a database connection, and telling Alembic what your “target” schema looks like (for autogenerate). Because it’s plain Python, you can make it do anything: read a URL from an environment variable, connect to multiple databases, skip logging setup, whatever your deployment needs.

Alembic ships four starter templates, and you pick the one that matches your project *once*, at `init` time:

Template	When to use it
<code>generic</code>	Sync SQLAlchemy, single database — the default
<code>async</code>	Async SQLAlchemy (asyncpg, aiosqlite, etc.) — this is the one TaskFlow uses
<code>pyproject</code>	Same as generic, but config lives in <code>pyproject.toml</code> instead of <code>alembic.ini</code>
<code>multidb</code>	Multiple databases from one Alembic environment

```
alembic init -t async alembic      # what TaskFlow's env.py is built from
```

`alembic.ini` — the part you’ll actually touch

Ninety percent of the file is comment-noise. The line that matters day one:

```
sqlalchemy.url = postgresql+psycopg2://scott:tiger@localhost/mydb
```

In real projects you almost never hardcode a URL here — you pull it from an environment variable inside `env.py` instead (critical for Docker; see section 6). When you do, this line becomes a fallback/placeholder and the real logic lives in `env.py`’s `run_migrations_online()`.

One config value worth turning on immediately in any real project:

```
file_template = %(year)d_%(month).2d_%(day).2d_%(hour).2d_%(minute).2d_%(rev)s_%(slug)s
```

Uncommenting this prefixes every revision filename with a timestamp, so `versions/` sorts chronologically in your file browser instead of by random hash. Small thing, saves real annoyance later.

Try It Yourself: In your scratch project, point `sqlalchemy.url` at a local SQLite file (`sqlite:///./lab.db`) instead of Postgres — no server needed for this section. Run `alembic current`. What does it print, and why does it print that on a brand-new, unmigrated database? (Hint: think about what table Alembic checks for first.)

3. Revisions: Writing Migrations by Hand

```
alembic revision -m "create account table"
```

This generates a file in `versions/` with a random revision id, an `upgrade()` and a `downgrade()` — both empty. You fill them in using **operation directives** from `alembic.op`:

```
def upgrade():
    op.create_table(
        "account",
        sa.Column("id", sa.Integer, primary_key=True),
        sa.Column("name", sa.String(50), nullable=False),
        sa.Column("description", sa.Unicode(200)),
    )

def downgrade():
    op.drop_table("account")
```

The `down_revision` variable at the top of the file is what links this script to the previous one — Alembic reads every file in `versions/`, follows the chain of `down_revision` pointers, and that's how it knows the order. This is also why revision *filenames* look like random hashes instead of `001`, `002`: the order comes from the pointer chain, not the filename, which means two developers can create revisions independently and merge them later without a numbering collision.

Running migrations

```
alembic upgrade head      # go to the newest revision
alembic upgrade +1       # move forward one step from wherever you are
alembic downgrade -1      # back one step
alembic downgrade base    # all the way back to nothing
alembic current           # what revision is the DB actually on right now
alembic history --verbose # the full chain, in order
```

`head` is the most common target you'll type. `+2`, `-1`, `ae10+2` (two steps past a specific revision) all work too, for when you're debugging a specific migration and don't want to jump all the way to head.

Try It Yourself: Create two revisions by hand (no autogenerate) — one that adds a table, one that adds a column to it. Upgrade to head, then downgrade all the way to `base`, then upgrade to head again. Watch the log output each time. What's the actual SQL Alembic ran to know where it currently stood, before you gave it any explicit revision number?

4. Autogenerate: Let Alembic Diff Your Models Against Your DB

Writing every `create_table` / `add_column` by hand gets old fast. Autogenerate compares your live database against your SQLAlchemy `MetaData` and drafts the migration for you.

Wire it up — in `env.py`, point `target_metadata` at your models' Base:

```
from myapp.models import Base
target_metadata = Base.metadata
```

Then:

```
alembic revision --autogenerate -m "added account table"
```

What it reliably detects

table/column add & remove · nullable changes · basic index and named-unique-constraint changes · basic FK changes · **named** CHECK constraints (new in 1.19)

What it detects only if you turn it on

- **Column type changes** — on by default since 1.12, but only catches *major* differences (e.g. `Numeric` → `String`); it won't reliably notice a length change from `VARCHAR(50)` to `VARCHAR(100)` in every backend.
- **Server default changes** — off by default (`compare_server_default=True` to enable), because it produces false positives easily.

What it can never detect — memorize this list, it will save you a bad migration someday

- **Table or column renames.** These show up as a drop + an add of a *different* named thing. If you rename `users` → `accounts`, autogenerate will happily write a migration that drops `users` and creates `accounts` — destroying every row. You must hand-edit renames using `op.rename_table()` / `op.alter_column(..., new_column_name=...)`.
- **Unnamed constraints.** If you didn't give a `UniqueConstraint` a `name=`, Alembic can't track it across revisions reliably. Fix this once, project-wide, with a naming convention (section 8) — don't fight it constraint by constraint.
- **Enum types on backends without native ENUM support** (it just sees a CHAR + CHECK and can't guess your intent).

This is the single most important habit to build: **autogenerate drafts, it does not decide.** Always open the generated file and read it before running `upgrade`. Treat the `--autogenerate` flag the same way you'd treat a linter's autofix — useful, not infallible.

Try It Yourself: Add a new column to one of your SQLAlchemy models, then rename an existing column in the model (don't touch the DB by hand). Run `--autogenerate`. Look at the generated file — does it correctly capture the rename, or does it do something destructive? Fix the generated migration by hand to use `alter_column(new_column_name=...)` instead.

5. SQLite Needs Special Handling: Batch Mode

Here’s a gap that bites almost everyone coming from Postgres/MySQL: **SQLite can’t ALTER TABLE most things**. No `DROP COLUMN`, no changing a column’s type, no adding a constraint to an existing column — SQLite’s `ALTER TABLE` support is deliberately minimal.

Alembic’s workaround is **batch mode**: instead of running `ALTER TABLE` directly, it creates a new temp table with the desired schema, copies the data over, drops the old table, and renames the temp table into place. You opt into it by wrapping operations in `batch_alter_table`:

```
def upgrade():
    with op.batch_alter_table("account") as batch_op:
        batch_op.add_column(sa.Column("email", sa.String(100)))
        batch_op.drop_column("legacy_flag")
```

For autogenerate to *emit* batch-wrapped operations automatically against SQLite, tell `context.configure()` in `env.py`:

```
context.configure(
    connection=connection,
    target_metadata=target_metadata,
    render_as_batch=True,  # <-- this line
)
```

This matters directly for the to-do app: since it’s on SQLite today, any migration beyond “add a table” or “add a nullable column” should go through batch mode, or it’ll fail outright with an `OperationalError` the moment you try to drop or alter a column.

Try It Yourself: On your SQLite scratch project, write a migration (by hand, no batch mode) that tries to drop a column. Run it and read the actual error SQLite/Alembic gives you. Then rewrite the same migration using `batch_alter_table` and confirm it succeeds.

6. Alembic + Docker

Nothing about Alembic itself changes in Docker — it’s still the same `alembic upgrade head`. What changes is **when and how you run that command**, since your database is a separate, ephemeral, network-addressed service instead of a file on disk.

Getting the connection string right

Inside Docker Compose, your app container reaches Postgres by **service name**, not `localhost`:

```
# docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: appdb
  web:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      DATABASE_URL: postgresql+asyncpg://postgres:devpass@db:5432/appdb
```

In `env.py`, read that from the environment instead of hardcoding `alembic.ini`:

```
import os
config.set_main_option(
    "sqlalchemy.url",
    os.environ.get("DATABASE_URL", config.get_main_option("sqlalchemy.url")),
)
```

This one change is what lets the exact same `alembic.ini` / `env.py` work identically on your laptop (`DATABASE_URL` unset → falls back to the `.ini` default) and inside the container (`DATABASE_URL` set by Compose).

Three ways to actually run migrations in a containerized setup

1. One-off exec, for local dev iteration — fastest feedback loop while you’re actively writing migrations:

```
docker compose exec web alembic upgrade head
docker compose exec web alembic revision --autogenerate -m "add table"
```

2. Entrypoint script, runs on every container start — good for small projects / solo dev, bad the moment you scale to multiple app replicas (they’d all race to migrate simultaneously):

```
#!/bin/sh
# entrypoint.sh
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

3. Dedicated one-shot migration service — the pattern that scales, and the one to reach for once you have more than one app replica or a real deploy pipeline:

```
services:
  migrate:
    build: .
    command: alembic upgrade head
    depends_on:
      db:
        condition: service_healthy
  web:
    build: .
    depends_on:
      migrate:
        condition: service_completed_successfully
```

The `migrate` service runs once, exits 0, and only then does `web` start — no race condition, and it’s a natural fit for CI/CD pipelines where you want a distinct “migrate” step with its own logs and exit code.

The `depends_on: condition:` trap

If you skip the `condition: service_healthy` / `service_completed_successfully` and just use plain `depends_on`, Compose only waits for the *container* to start, not for Postgres to actually be ready to accept connections. Alembic will then fail with a connection-refused error in a

fresh `docker compose up`. Give the `db` service a `healthcheck:` (e.g. `pg_isready`) and reference it as shown above.

Try It Yourself: Take your scratch project, containerize it with a `docker-compose.yml` (Postgres + a throwaway Python service), and get pattern #3 working — a `migrate` service that runs once and exits, gating a `web` service that only starts after. Kill the DB container mid-migration on purpose and watch how Alembic reports the failure.

7. Async SQLAlchemy: Where Alembic’s Sync Nature Shows

Alembic itself doesn’t have an async API — its internal migration engine is synchronous. When you’re on async SQLAlchemy (asyncpg, aiosqlite), `env.py`’s `run_migrations_online()` has to bridge the gap using `run_sync`:

```
import asyncio
from sqlalchemy.ext.asyncio import async_engine_from_config

def do_run_migrations(connection):
    context.configure(connection=connection, target_metadata=target_metadata)
    with context.begin_transaction():
        context.run_migrations()

async def run_async_migrations():
    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )
    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)
    await connectable.dispose()

def run_migrations_online():
    asyncio.run(run_async_migrations())
```

This is exactly what the `async` init template gives you for free — worth reading through once now that you know what each piece is doing, rather than treating it as a black box Kimi/Claude generated for you.

Note the `poolclass=pool.NullPool`. Migrations are short-lived, one-shot processes — you don’t want connection pooling overhead or lingering pooled connections when the process exits right after. This is a deliberate choice, not a default you should “clean up.”

8. The Trap Almost Everyone Hits: Unnamed Constraints

Covered briefly above, but it deserves its own section because it's the #1 source of “why did autogenerate just try to drop and recreate my entire constraint for no reason” confusion.

SQLAlchemy, by default, lets the database auto-name constraints (Postgres will call an index something like `account_name_key`). Because that name isn't something *your* metadata specified, Alembic can't reliably match “the constraint the DB has” to “the constraint your model wants” across runs — so autogenerate keeps flip-flopping on it.

Fix it once, project-wide, with a naming convention on your metadata:

```
convention = {
    "ix": "ix_(column_0_label)s",
    "uq": "uq_(table_name)s_(column_0_name)s",
    "ck": "ck_(table_name)s_(constraint_name)s",
    "fk": "fk_(table_name)s_(column_0_name)s_(referred_table_name)s",
    "pk": "pk_(table_name)s",
}
metadata = MetaData(naming_convention=convention)
Base = declarative_base(metadata=metadata)
```

Do this early in a project's life — retrofitting it means an autogenerate run that “renames” every existing constraint in your database, which is noisy but harmless (rename operations, not data loss) as long as you review the diff before applying it.

9. Offline Mode: Generating SQL Without a Live Connection

`alembic upgrade head --sql` doesn't touch a database at all — it prints the raw SQL that *would* run, to stdout (or a file). This exists for environments where your migration tooling isn't allowed direct database access — a DBA reviews and runs the SQL by hand, or a production deploy pipeline that requires SQL sign-off before execution.

```
alembic upgrade head --sql > migration.sql
```

Worth knowing exists even if you don't need it day-to-day on personal projects — it comes up constantly in real-job contexts once “someone else's prod database” enters the picture.

10. Branches, in Brief

If two people create a revision from the same parent independently (common on a team, or if you’re maintaining parallel feature branches), you get two `head`s. Alembic won’t silently pick one:

```
alembic heads          # lists all current head revisions
alembic merge -m "merge heads" head1 head2  # creates a merge revision joining them
```

You won’t hit this solo very often, but the moment you and a collaborator both add migrations on separate branches and merge your git branches, `alembic upgrade head` will error with “multiple heads” until you run `alembic merge`. Good to recognize the error message rather than be baffled by it.

11. Command Cheat Sheet

Command	What it does
<code>alembic init -t async alembic</code>	Scaffold a new async migration environment
<code>alembic revision -m "msg"</code>	New empty revision
<code>alembic revision --autogenerate -m "msg"</code>	New revision, drafted from model/DB diff
<code>alembic upgrade head</code>	Migrate to the latest revision
<code>alembic upgrade +1 / downgrade -1</code>	Step forward/back one revision
<code>alembic downgrade base</code>	Revert everything
<code>alembic current</code>	What revision is the DB on
<code>alembic history --verbose</code>	Full revision chain
<code>alembic heads</code>	List current head(s) — >1 means unmerged branches
<code>alembic merge -m "msg" rev1 rev2</code>	Merge divergent heads
<code>alembic check</code>	Exit non-zero if models and DB have diverged (no file generated) — great for CI
<code>alembic upgrade head --sql</code>	Print SQL instead of executing (offline mode)
<code>alembic stamp head</code>	Mark DB as being at a revision <i>without</i> running migrations — used when you <code>create_all()</code> 'd a fresh DB and just need to seed <code>alembic_version</code>

12. `alembic check` in CI — a Feature Worth Adopting Now

One command that's easy to miss and genuinely useful once you're pushing to a shared repo: `alembic check` runs the same model-vs-database comparison as `--autogenerate`, but generates nothing — it just tells you, with an exit code, whether your models have drifted from your last migration. Wire it into a pre-commit hook or CI step and you'll never again merge a model change that silently has no matching migration:

```
alembic check
# No new upgrade operations detected.  <- exit 0, all good
# FAILED: New upgrade operations detected: [...]  <- exit 1, you forgot a migration
```

13. Troubleshooting Decision Tree

“Target database is not up to date” on upgrade → you have unresolved multiple heads, or the DB is stamped at a revision that doesn’t exist in your `versions/` folder (someone deleted a migration file). Run `alembic heads` and `alembic history` to see the actual chain.

Autogenerate detects changes you didn’t make, every single run → almost always unnamed constraints (section 8), or a type comparison false-positive on a custom type — set `compare_type` to a callable that returns `None` (defer to default) for the cases that are actually fine.

Migration works locally but fails in Docker with “connection refused” → your `depends_on` isn’t waiting for Postgres to be *ready*, just for the container to *start*. Add a proper healthcheck (section 6).

SQLite migration fails with `OperationalError near "ALTER"` → you need batch mode (section 5). This basically never happens on Postgres/MySQL, which is why it surprises people who developed against Postgres and only hit SQLite later (or vice versa, like moving from the to-do app’s SQLite to TaskFlow’s Postgres).

`alembic revision --autogenerate` **produces an empty migration when you know something changed** → `target_metadata` isn’t actually pointing at the right `Base.metadata` in `env.py`, or the model with the change was never imported anywhere `env.py` reaches (SQLAlchemy only knows about models that have been imported into a shared `Base`).

14. Suggested Progression

Given where you already are (TaskFlow’s async Postgres setup is running, the to-do app’s SQLite migrations are still light), here’s an order that builds on what you already have rather than starting from zero:

1. Do every “Try It Yourself” in this guide against the scratch project — SQLite first (sections 1-5), no Docker, so you can see raw error messages without a container layer in between.
 2. Retrofit the naming convention (section 8) into the to-do app’s models, run autogenerate, and read the (harmless) rename-diff it produces.
 3. Deliberately break the to-do app’s SQLite migrations with a column drop, hit the batch-mode error for real, then fix it — this is the exercise most people skip and then get burned by later.
 4. Containerize the to-do app with the pattern-3 `migrate` service from section 6, even though it’s “just” SQLite — the Docker orchestration lesson matters independent of which database sits behind it.
 5. Go back to TaskFlow and read its existing `env.py` line by line against section 7 — you’ll likely recognize every piece now instead of it being mentor-provided boilerplate.
 6. Add `alembic check` as a step you run before every commit that touches a model, going forward.
-

Sources: this guide was checked directly against the official Alembic documentation (alembic.sqlalchemy.org) at the 1.19.x release, current as of August 2026 — not reconstructed from memory.